FreeRTOS Switch / Shared Variable / UART Assignment

Code reference guide with API documentation locations

Target assumptions used in this guide: STM32F103C8 (Blue Pill), switch on PA0 / EXTI0, USART1 on PA9 (TX) and PA10 (RX), 115200 baud, 8-N-1. If your wiring or MCU differs, change the GPIO/UART configuration accordingly.

Documentation convention: HAL page numbers below refer to ST user manual UM1850, Rev. 3. The printed page number shown as “page x/1208” is used (not the PDF viewer's page index). FreeRTOS APIs are linked to their official API reference pages.

1. Where the APIs come from

API / function	Documentation location
HAL_GPIO_Init	ST UM1850, GPIO driver, section 20.2.3 / detailed description; printed page 226. GPIO EXTI configuration modes and pull settings are also explained on page 27.
HAL_GPIO_EXTI_IRQHandler	ST UM1850, GPIO driver, section 20.2.4 (GPIO I/O operations); also page 27 explains that the EXTI IRQ handler must call this function.
HAL_GPIO_EXTI_Callback	ST UM1850, page 27: user implements this callback when EXTI interrupt mode is used. GPIO I/O API listing is in section 20.2.4.
HAL_UART_Init	ST UM1850, UART driver, section 38.2.7, printed page 554.
HAL_UART_Transmit	ST UM1850, UART driver, section 38.2 (I/O operation functions); see the UART transmit API description in the pages following the initialization description (around printed pages 556–557).
HAL_UART_MspInit	ST UM1850, UART driver, section 38.2: low-level UART clock and GPIO setup is performed through the user-implemented MSP init function; see the UART initialization procedure around pages 550 and 554.
xTaskCreate	FreeRTOS official API: https://freertos.org/Documentation/02-Kernel/04-API-references/01-Task-creation/01-xTaskCreate
vTaskDelayUntil	FreeRTOS official API: https://freertos.org/Documentation/02-Kernel/04-API-references/02-Task-control/03-vTaskDelayUntil
vTaskDelay	FreeRTOS official API: https://freertos.org/Documentation/02-Kernel/04-API-references/02-Task-control/01-vTaskDelay
xEventGroupCreate	FreeRTOS official API: https://freertos.org/Documentation/02-Kernel/04-API-references/12-Event-groups-or-flags/01-xEventGroupCreate
xEventGroupSetBitsFromISR	FreeRTOS official API: https://www.freertos.org/Documentation/02-Kernel/04-API-references/12-Event-groups-or-flags/03-xEventGroupSetBitsFromISR
xEventGroupWaitBits	FreeRTOS official API: https://freertos.org/Documentation/02-Kernel/04-API-references/12-Event-groups-or-flags/04-xEventGroupWaitBits
xQueueCreate	FreeRTOS official API: https://freertos.org/Documentation/02-Kernel/04-API-references/06-Queues/01-xQueueCreate
xQueueSend	FreeRTOS official API: https://freertos.org/Documentation/02-Kernel/04-API-references/06-Queues/07-xQueueSend
xQueueReceive	FreeRTOS official API: https://freertos.org/Documentation/02-Kernel/04-API-references/06-Queues/10-xQueueReceive
xSemaphoreCreateMutex / xSemaphoreTake / xSemaphoreGive	FreeRTOS official Semaphore and Mutex API: https://freertos.org/Documentation/02-Kernel/04-API-references/11-Semaphore-and-Mutexes
vTaskStartScheduler	FreeRTOS official API: https://freertos.org/Documentation/02-Kernel/04-API-references/03-Task-utilities/04-vTaskStartScheduler

2. main.c integration

Keep the UART handle at file scope (not inside MX_USART1_UART_Init). The FreeRTOS source file accesses it through the extern declaration in main.h.

```
/* main.c */
#include "main.h"
#include "FreeRTOS.h"     /* FreeRTOS kernel types/macros */
#include "task.h"         /* vTaskStartScheduler() */

UART_HandleTypeDef huart1; /* Global handle; visible to FreeRTOS.c */

int main(void)
{
    HAL_Init();
    SystemClock_Config();

    MX_GPIO_Init();       /* Configure PA0 EXTI and any required GPIO */
    MX_USART1_UART_Init();/* HAL_UART_Init() initializes USART1 */

    MX_FREERTOS_Init();   /* Creates queue, event group, mutex and tasks */
    vTaskStartScheduler();/* FreeRTOS API: starts task scheduling */

    while (1) { }
}
```

In main.h, declare the functions and UART handle (ensure the declarations are inside the header guard):

```
/* main.h */
extern UART_HandleTypeDef huart1;
void MX_FREERTOS_Init(void);
```

GPIO / EXTI setup (PA0, falling edge)

```
/* Inside MX_GPIO_Init() after enabling GPIOA and AFIO clocks */
GPIO_InitTypeDef GPIO_InitStruct = {0};

__HAL_RCC_GPIOA_CLK_ENABLE();
__HAL_RCC_AFIO_CLK_ENABLE();

GPIO_InitStruct.Pin  = GPIO_PIN_0;
GPIO_InitStruct.Mode = GPIO_MODE_IT_FALLING;
GPIO_InitStruct.Pull = GPIO_PULLUP;
HAL_GPIO_Init(GPIOA, &GPIO_InitStruct); /* ST UM1850: GPIO init, p.226;
                                           EXTI modes/pull config, p.27 */

HAL_NVIC_SetPriority(EXTI0_IRQn, 5, 0);
HAL_NVIC_EnableIRQ(EXTI0_IRQn); /* NVIC configuration for EXTI, UM1850 p.226
                                   (GPIO how-to-use procedure) */
```

In stm32f1xx_it.c, ensure the EXTI0 handler calls the HAL handler:

```
void EXTI0_IRQHandler(void)
{
    HAL_GPIO_EXTI_IRQHandler(GPIO_PIN_0); /* ST UM1850 p.27; GPIO I/O API,
                                             section 20.2.4 */
}
```

USART1 initialization

```
/* main.c: one definition only */
static void MX_USART1_UART_Init(void)
{
    huart1.Instance          = USART1;
    huart1.Init.BaudRate     = 115200;
    huart1.Init.WordLength   = UART_WORDLENGTH_8B;
    huart1.Init.StopBits     = UART_STOPBITS_1;
    huart1.Init.Parity       = UART_PARITY_NONE;
    huart1.Init.Mode         = UART_MODE_TX_RX;
    huart1.Init.HwFlowCtl    = UART_HWCONTROL_NONE;
    huart1.Init.OverSampling = UART_OVERSAMPLING_16;

    HAL_UART_Init(&huart1); /* ST UM1850, section 38.2.7, p.554 */
}
```

HAL_UART_MspInit() must enable the USART1 and GPIOA clocks and configure PA9 as alternate-function push-pull and PA10 as input. In a Cube-generated project this function may already exist in stm32f1xx_hal_msp.c; do not define it twice.

3. FreeRTOS.c — shared definitions and object creation

```
#include "main.h"
#include "FreeRTOS.h"
#include "task.h"
#include "queue.h"
#include "event_groups.h"
#include "semphr.h"
#include <stdio.h>
#include <stdint.h>

typedef enum {
    SOURCE_COUNTER = 0,
    SOURCE_SHARED
} DataSource_t;

typedef struct {
    DataSource_t source;
    uint32_t value;
} UARTMessage_t;

volatile uint32_t counter = 0;
uint32_t shared = 100;

static QueueHandle_t UARTData;
static EventGroupHandle_t switchEventGroup;
static SemaphoreHandle_t sharedMutex;

#define SWITCH_PRESSED_BIT (1U << 0)

static void CalcSwitch1(void *argument);
static void IncreaseVar(void *argument);
static void DecreaseVar(void *argument);
static void UARTTask(void *argument);

void MX_FREERTOS_Init(void)
{
    UARTData = xQueueCreate(16, sizeof(UARTMessage_t));
    switchEventGroup = xEventGroupCreate();
    sharedMutex = xSemaphoreCreateMutex();

    configASSERT(UARTData != NULL);
    configASSERT(switchEventGroup != NULL);
    configASSERT(sharedMutex != NULL);

    xTaskCreate(CalcSwitch1, "CalcSwitch1", 256, NULL,
                tskIDLE_PRIORITY + 1, NULL);
    xTaskCreate(IncreaseVar, "IncreaseVar", 256, NULL,
                tskIDLE_PRIORITY + 1, NULL);
    xTaskCreate(DecreaseVar, "DecreaseVar", 256, NULL,
                tskIDLE_PRIORITY + 1, NULL);
    xTaskCreate(UARTTask, "UARTTask", 256, NULL,
                tskIDLE_PRIORITY + 1, NULL);
}
```

API notes: xQueueCreate(), xEventGroupCreate(), xSemaphoreCreateMutex(), and xTaskCreate() are FreeRTOS kernel APIs (links in section 1). The stack depth passed to xTaskCreate is in words, not bytes.

4. FreeRTOS.c — task implementations

```
static void CalcSwitch1(void *argument)
{
    UARTMessage_t message;
    (void)argument;

    for (;;)
    {
        EventBits_t bits = xEventGroupWaitBits(
            switchEventGroup,
            SWITCH_PRESSED_BIT,
            pdTRUE,          /* clear bit on exit */
            pdFALSE,         /* wait for any requested bit */
            pdMS_TO_TICKS(2) /* timeout: 2 ticks */
        ); /* FreeRTOS API: xEventGroupWaitBits() */

        if ((bits & SWITCH_PRESSED_BIT) != 0U)
        {
            uint32_t value = counter;
            message.source = SOURCE_COUNTER;
            message.value = value * 10U; /* keep original counter unchanged */

            xQueueSend(UARTData, &message, portMAX_DELAY);
            /* FreeRTOS API: xQueueSend() */
        }
    }
}

static void IncreaseVar(void *argument)
```

```
{
    TickType_t lastWakeTime = xTaskGetTickCount();
    UARTMessage_t message;
    (void)argument;

    for (;;)
    {
        vTaskDelayUntil(&lastWakeTime, pdMS_TO_TICKS(10));
        /* FreeRTOS API: vTaskDelayUntil() */

        xSemaphoreTake(sharedMutex, portMAX_DELAY);
        /* FreeRTOS API: xSemaphoreTake() */

        shared++;
        if (shared > 200U) shared = 100U;
        message.source = SOURCE_SHARED;
        message.value = shared;

        xSemaphoreGive(sharedMutex);
        /* FreeRTOS API: xSemaphoreGive() */

        xQueueSend(UARTData, &message, portMAX_DELAY);
    }
}

static void DecreaseVar(void *argument)
{
    TickType_t lastWakeTime = xTaskGetTickCount();
    UARTMessage_t message;
    (void)argument;

    for (;;)
    {
        vTaskDelayUntil(&lastWakeTime, pdMS_TO_TICKS(12));
        /* FreeRTOS API: vTaskDelayUntil() */

        xSemaphoreTake(sharedMutex, portMAX_DELAY);

        if (shared < 100U) shared = 200U;
        else shared--;

        message.source = SOURCE_SHARED;
        message.value = shared;

        xSemaphoreGive(sharedMutex);
        xQueueSend(UARTData, &message, portMAX_DELAY);
    }
}

static void UARTTask(void *argument)
{
    UARTMessage_t message;
    char txBuffer[40];
    (void)argument;

    for (;;)
    {
        vTaskDelay(pdMS_TO_TICKS(5));
        /* FreeRTOS API: vTaskDelay() */

        if (xQueueReceive(UARTData, &message, 0) == pdPASS)
        {
            int length;

            if (message.source == SOURCE_COUNTER)
                length = snprintf(txBuffer, sizeof(txBuffer),
                                  "Counter: %lu\\r\\n",
                                  (unsigned long)message.value);
            else
                length = snprintf(txBuffer, sizeof(txBuffer),
                                  "Shared: %lu\\r\\n",
                                  (unsigned long)message.value);

            if (length > 0)
            {
                HAL_UART_Transmit(&huart1,
                    (uint8_t *)txBuffer,
                    (uint16_t)length,
                    HAL_MAX_DELAY);
                /* ST HAL API: UM1850 UART driver, section 38.2,
                   transmit I/O operation (around pp.556–557). */
            }
        }
    }
}
```

This follows the assignment's stated periods assuming configTICK_RATE_HZ = 1000 Hz (1 tick = 1 ms). The shared variable is read/updated while holding the mutex. The UART task sends at most one queued message each time it wakes.

5. FreeRTOS.c — switch callback and stack hook

```
void HAL_GPIO_EXTI_Callback(uint16_t GPIO_Pin)
{
    if (GPIO_Pin == GPIO_PIN_0)
    {
        counter++;
        if (counter >= 9U) counter = 0U;

        if (switchEventGroup != NULL)
        {
            BaseType_t higherPriorityTaskWoken = pdFALSE;

            BaseType_t result = xEventGroupSetBitsFromISR(
                switchEventGroup,
                SWITCH_PRESSED_BIT,
                &higherPriorityTaskWoken);
            /* FreeRTOS API: xEventGroupSetBitsFromISR().
               It defers the event-bit operation to the timer daemon task. */

            if (result == pdPASS)
                portYIELD_FROM_ISR(higherPriorityTaskWoken);
        }
    }
}
/* ST HAL callback: UM1850 p.27 (EXTI callback requirement);
   GPIO I/O API section 20.2.4. */

/* Required only when configCHECK_FOR_STACK_OVERFLOW is enabled. */
void vApplicationStackOverflowHook(TaskHandle_t task, char *taskName)
{
    (void)task;
    (void)taskName;
    taskDISABLE_INTERRUPTS();
    for (;;) { }
}
```

Important FreeRTOSConfig.h settings: configTICK_RATE_HZ must be 1000 for the 1 ms tick assumption; configUSE_MUTEXES must be 1. Because xEventGroupSetBitsFromISR() uses the timer service task, include timers.c in the build, set configUSE_TIMERS to 1, and set INCLUDE_xTimerPendFunctionCall to 1. Configure a valid timer task stack, priority, and timer command queue length.

The ISR event bit is a flag, not a counter: multiple presses occurring before CalcSwitch1 clears the bit can be coalesced into one event. Mechanical switch debouncing may also be needed.

6. API checklist for the report

Source	API used in this project	Purpose
ST HAL	HAL_GPIO_Init	Configures PA0 as EXTI input; also used for GPIO pin configuration.
ST HAL	HAL_GPIO_EXTI_IRQHandler	Checks/clears EXTI pending state and calls the user callback.
ST HAL	HAL_GPIO_EXTI_Callback	User callback executed after the external interrupt.
ST HAL	HAL_UART_Init	Initializes USART1 from UART handle configuration.
ST HAL	HAL_UART_Transmit	Sends the formatted message to the PC over UART.
FreeRTOS	xTaskCreate, vTaskStartScheduler	Creates tasks and starts the scheduler.
FreeRTOS	xEventGroupCreate, xEventGroupSetBitsFromISR, xEventGroupWaitBits	Creates and uses the switch event flag.
FreeRTOS	xQueueCreate, xQueueSend, xQueueReceive	Transfers typed messages from tasks to UARTTask.
FreeRTOS	xSemaphoreCreateMutex, xSemaphoreTake, xSemaphoreGive	Protects shared from concurrent task access.
FreeRTOS	vTaskDelay, vTaskDelayUntil, xTaskGetTickCount	Implements periodic task timing.

7. Source documents

STMicroelectronics, UM1850 Rev. 3 — Description of STM32F1 HAL and low-layer drivers (official PDF):

Open ST UM1850 PDF

FreeRTOS Kernel API reference (official website):

Open FreeRTOS API reference index

This guide documents the implementation pattern and API locations. Check your installed HAL/FreeRTOS versions and your actual board pinout before flashing. The PA0/USART1 pin choices are assumptions for the example, not confirmation of your particular wiring.